

AI-Powered Fitness Coach

Complete Operations, Integration & Deployment Guide

Version: 1.0.0 | **Date:** 12 September 2026 | **Stack:** FastAPI + React/Vite + MediaPipe + Supabase + Redis/Celery

Conventions used in this guide: **[PROJECT]** = applies directly to this repository as implemented. **[REFERENCE]** = general production guidance to adopt when needed. All commands assume Windows PowerShell unless marked otherwise; Linux/macOS equivalents differ only in activation and path syntax.

Table of Contents

Table of Contents	2
1. Local Execution & Verification [PROJECT]	4
1.1 Prerequisites	4
1.2 Repository layout (abridged)	4
1.3 Backend setup	4
1.4 Frontend setup	4
1.5 Environment configuration	4
1.6 Run order (local, without Docker)	5
1.7 Verification methods	5
1.8 Troubleshooting startup failures [PROJECT]	6
2. Docker Integration [PROJECT + REFERENCE]	6
2.1 Current Dockerfiles (as shipped)	6
2.2 Production backend Dockerfile (multi-stage, copy-paste) [REFERENCE]	6
2.3 docker-compose.yml (as shipped, annotated)	7
2.4 Build, run, debug	7
2.5 Networking, volumes, env patterns	7
3. Supabase Integration & Database Architecture [PROJECT + REFERENCE]	7
3.1 Project initialization & connection [PROJECT]	7
3.2 Schema design [PROJECT]	8
3.3 Row Level Security [PROJECT]	8
3.4 Query cookbook (copy-paste) [PROJECT queries; REFERENCE patterns]	9
3.5 Authentication [PROJECT state + REFERENCE path]	9
3.6 Storage [PROJECT]	10
3.7 Edge Functions [REFERENCE - not used by this project]	10
4. Platform-Specific Production Deployment	10
4.1 Vercel - frontend only [PROJECT-applicable]	10
4.2 Render - full stack [PROJECT-applicable]	10
4.3 Alternatives [REFERENCE]	11
5. System Limitations & Constraints	11
5.1 Technical boundaries [PROJECT]	11
5.2 Platform restrictions [REFERENCE]	12
5.3 Rate limiting & quotas [PROJECT + REFERENCE]	12
5.4 Security checklist [PROJECT + REFERENCE]	12
6. Database Operations & Query Optimization	12
6.1 Essential queries [PROJECT]	12
6.2 Performance [PROJECT + REFERENCE]	12

6.3 Zero-downtime migrations [REFERENCE]	13
6.4 Backups & recovery [REFERENCE]	13
7. Monitoring, Logging & Maintenance	13
7.1 Structured logging [PROJECT]	13
7.2 Error tracking [REFERENCE]	13
7.3 Uptime & alerting [PROJECT + REFERENCE]	13
7.4 CI/CD pipeline [REFERENCE - starter, adapt paths]	14
Appendix A - Endpoint reference [PROJECT]	14
Appendix B - Measured ML baselines [PROJECT]	14

1. Local Execution & Verification [PROJECT]

1.1 Prerequisites

Requirement	Version used by this project	Notes
Python	3.12 (Dockerfile uses python:3.12-slim; README supports 3.10+)	MediaPipe + TensorFlow wheels required
Node.js + npm	Node 20 (Dockerfile), npm 10	Frontend build uses node:20-alpine
Redis	7-alpine (docker-compose)	Broker + result backend for Celery
Git, PowerShell	any recent	Windows 10/11 assumed
Docker (optional)	20.10+	Only needed for Section 2

1.2 Repository layout (abridged)

```
AIML_Project/  
  backend/app/          # FastAPI: main.py, api/v1/*, core/*, services/*, ml/*, workers/*  
  backend/Dockerfile  
  frontend/src/         # React + TS + Vite (pages, components, services/api.ts)  
  frontend/Dockerfile   # multi-stage node build -> nginx  
  frontend/nginx.conf   # SPA fallback + /api/ reverse proxy to backend:8000  
  supabase/migrations/001_initial_schema.sql  
  scripts/              # extract_poses, compile_features, evaluate_models  
  models/               # random_forest, xgboost, lstm artifacts  
  data/                 # raw uploads, interim poses, processed splits  
  docker-compose.yml    # redis + backend + worker + frontend
```

1.3 Backend setup

```
cd D:\AIML_Project  
python -m venv .venv  
.\venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

Key pinned floors from **requirements.txt**: fastapi>=0.111, uvicorn[standard]>=0.29, pydantic>=2.7, pydantic-settings>=2.2.1, opencv-python-headless>=4.8, mediapipe>=0.10.14, numpy>=1.26,<2.6, pandas>=2.2, scikit-learn>=1.4, xgboost>=2.0, torch>=2.2, reportlab>=4.2, redis>=5, celery>=5.3, supabase>=2.4, pytest>=8, httpx>=0.27, pyjwt>=2.8, python-multipart.

1.4 Frontend setup

```
cd D:\AIML_Project\frontend  
npm ci          # reproducible install from package-lock.json  
npm run dev     # Vite dev server -> http://localhost:5173
```

Frontend deps: react/react-dom 18.3.1, lucide-react 0.383; dev: vite 5.2.11, typescript 5.4.5. Production build: **npm run build** (runs tsc then vite build). Type check only: **npx tsc --noEmit**.

1.5 Environment configuration

Copy **.env.example** to **.env** at the repo root (never commit **.env**; it is git-ignored). Without Supabase keys the backend automatically uses the local SQLite fallback at **backend/data/fitness_coach_local.db**.

```
copy .env.example .env  
# then edit: SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY (Section 3)  
# REDIS_URL / CELERY_* default to redis://localhost:6379 and work out of the box
```

Variable	Default	Purpose
SUPABASE_URL / ANON_KEY / SERVICE_ROLE_KEY	(empty = local SQLite mode)	Cloud DB/auth/storage; service key backend-only
REDIS_URL / CELERY_BROKER_URL	redis://localhost:6379/0	Task queue
CELERY_RESULT_BACKEND	redis://localhost:6379/1	Task results
MAX_VIDEO_SIZE_MB / MAX_VIDEO_DURATION_SECONDS	500 / 300	Upload guardrails
SCORE_WEIGHT_*	0.35 / 0.25 / 0.20 / 0.20	Form / ROM / consistency / completion
AUTH_RATE_LIMIT / UPLOAD_RATE_LIMIT	10/minute / 5/hour	Abuse guardrails (see 5.4)

1.6 Run order (local, without Docker)

```
# Terminal 1 - Redis (or: docker run -p 6379:6379 redis:7-alpine)
redis-server
# Terminal 2 - API (from repo root, venv active)
uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload
# Terminal 3 - Celery worker (from repo root; PYTHONPATH picks up backend/)
celery -A app.workers.celery_app.celery_app worker --loglevel=info --concurrency=2
# Terminal 4 - Frontend
cd frontend; npm run dev
```

The worker command above matches docker-compose.yml. Run it with working directory = backend/ OR repo root with PYTHONPATH=backend; the current compose file uses /app with PYTHONPATH=/app.

1.7 Verification methods

1.7.1 Health endpoints [PROJECT]

```
curl http://localhost:8000/health # {"status":"healthy",...}
curl http://localhost:8000/ # links to /docs, /health, /api/v1
# Interactive API docs: http://localhost:8000/docs (Swagger) | /redoc
```

1.7.2 Log monitoring [PROJECT]

Backend emits structured JSON logs (see **backend/app/core/logging.py**) with timestamp, level, logger, module, and correlation fields **request_id** / **analysis_id** / **stage** when supplied. Set **LOG_LEVEL=DEBUG** and **LOG_FORMAT=text** for human-readable local logs. Never log passwords, tokens, or service-role keys (the formatter redacts sensitive keys).

```
# Watch worker progress for an analysis (PowerShell)
docker logs -f fitness_celery_worker # compose name
# or local: worker stdout already streams stage transitions + rep counts
```

1.7.3 Automated test suites [PROJECT]

```
python -m pytest tests/ -v # backend: 14 tests (API, features, rep counters, scoring)
cd frontend; npx tsc --noEmit # frontend type check (must be clean)
```

1.7.4 Manual end-to-end validation [PROJECT]

```
# 1. Upload a sample video (use Dataset/Dataset2/my_test_video_1/<exercise>/*.mp4)
# 2. POST /api/v1/videos/upload -> { id }
# 3. POST /api/v1/videos/{id}/analyze -> { analysis_id }
# 4. Stream progress: GET /api/v1/analysis/{analysis_id}/stream (SSE)
# 5. Fetch result: GET /api/v1/analysis/{analysis_id}
# 6. Download report: GET /api/v1/reports/{analysis_id}/pdf
```

Reference result on project hardware: a squat clip completes in ~24 s, yields 7 reps with per-rep ROM/duration, and an overall score with grade. A push-up clip yields 9 reps. Zero reps with good visibility indicates counter-threshold issues (see troubleshooting).

1.8 Troubleshooting startup failures [PROJECT]

Symptom	Likely cause	Fix
uvicorn: ModuleNotFoundError app	Wrong working dir / PYTHONPATH	Run from repo root; backend/ must be importable
MediaPipe import DLL error (Windows)	Missing VS C++ runtime	Install MSVC redistributable; use Python 3.12 64-bit
Port 8000 already in use	Stale uvicorn	netstat -ano findstr :8000 then taskkill /PID /F
Port 6379 refused	Redis not running	Start redis-server or the compose redis service
Celery tasks stuck PENDING	Worker using different broker DB	Align CELERY_BROKER_URL (.env vs compose env)
TensorFlow oneDNN / protobuf warnings	Benign CPU-info logs	Ignore; or set TF_ENABLE_ONEDNN_OPTS=0
Frontend blank page, API 404	nginx /api/ proxy vs dev server	In dev, Vite proxies /api/; in compose, nginx forwards to backend:8000
.env changes ignored	pydantic-settings cached at import	Restart uvicorn/worker; check file is repo-root .env

2. Docker Integration [PROJECT + REFERENCE]

2.1 Current Dockerfiles (as shipped)

backend/Dockerfile (single-stage, python:3.12-slim + libgl/libglib for OpenCV/MediaPipe, PYTHONPATH=/app, serves on 8000). **frontend/Dockerfile** (multi-stage: node:20-alpine build -> nginx:alpine serving dist/ on 80, with SPA fallback and /api/ reverse proxy incl. SSE-friendly buffering off and long read timeout).

2.2 Production backend Dockerfile (multi-stage, copy-paste) [REFERENCE]

Slimmer, layer-cached production image. Replace backend/Dockerfile with this when image size and rebuild speed matter:

```
FROM python:3.12-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
RUN apt-get update && apt-get install -y --no-install-recommends libgl1 libglib2.0-0 \
  && rm -rf /var/lib/apt/lists/*
WORKDIR /app
FROM base AS deps
COPY requirements.txt .
RUN pip install --prefix=/install -r requirements.txt
FROM base AS runtime
COPY --from=deps /install /usr/local
COPY backend/ /app/
COPY models/ /app/models/
ENV PYTHONPATH=/app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1"]
```

Keep --workers 1 for the API when MediaPipe/TensorFlow share one process; scale via more containers, not workers, to avoid OOM (each worker loads ~1 GB of ML libraries).

2.3 docker-compose.yml (as shipped, annotated)

```
services:
  redis:      image redis:7-alpine, port 6379, volume redis_data, healthcheck redis-cli ping
  backend:    build ./backend/Dockerfile, port 8000, env REDIS_URL + CELERY_*, volumes ./data
  ./models
  worker:     same image, command celery ... --concurrency=2, same volumes
  frontend:   build ./frontend/Dockerfile, port 5173:80, depends_on backend
```

Inter-service DNS: compose creates hostnames **redis**, **backend**, **worker**, **frontend**; hence **REDIS_URL=redis://redis:6379/0** inside containers and nginx **proxy_pass http://backend:8000**. Volumes **./data** and **./models** are bind-mounted so uploads, the SQLite fallback DB, and model artifacts persist across restarts. Inject secrets via an **env_file**: **.env** entry or a secrets manager - never bake keys into images.

2.4 Build, run, debug

```
docker compose up --build      # build + start all services
docker compose up -d backend worker # detached, subset
docker compose ps              # status
docker compose logs -f backend  # follow API logs (JSON lines)
docker compose logs -f worker   # follow Celery/ML logs
docker compose exec backend sh   # shell inside API container
docker compose exec backend python -m pytest tests/ -q
docker compose down             # stop; add -v to drop redis_data too (destroys queued tasks)
```

2.5 Networking, volumes, env patterns

Concern	Project setup	Production hardening [REFERENCE]
Service discovery	Compose DNS (backend, redis)	Same on Render/Railway private networks; use internal URLs
Frontend->API	nginx /api/ proxy (same origin, no CORS)	Keep same-origin proxying; restrict CORS_ORIGINS to the real domain
Persistence	Bind mounts ./data ./models	Named volumes or managed disks (Render disk, Fly volume)
Env injection	environment: block in compose	env_file + platform secret store; rotate SUPABASE_SERVICE_ROLE_KEY quarterly
Health gating	redis healthcheck; backend depends_on	Add backend healthcheck (Dockerfile above) + depends_on condition

3. Supabase Integration & Database Architecture [PROJECT + REFERENCE]

3.1 Project initialization & connection [PROJECT]

```
# 1. Create project at https://supabase.com/dashboard -> Settings -> API: copy URL + anon key + service_role key
# 2. SQL Editor -> paste supabase/migrations/001_initial_schema.sql -> Run
# 3. Repo root .env:
SUPABASE_URL=https://xyzcompany.supabase.co
SUPABASE_ANON_KEY=eyJ... # safe for browser use
SUPABASE_SERVICE_ROLE_KEY=eyJ... # BACKEND ONLY - bypasses RLS
```

Backend client: **supabase>=2.4 (supabase-py)** via **backend/app/services/supabase_client.py**. If keys are absent it falls back to local SQLite with an identical table-proxy API, so all service code runs unchanged offline.

```
# Python (backend pattern)
from supabase import create_client
db = create_client(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY) # server side
# Browser (REFERENCE - frontend does not use Supabase directly today)
import { createClient } from '@supabase/supabase-js'
export const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_ANON_KEY)
```

3.2 Schema design [PROJECT]

Authoritative DDL: [supabase/migrations/001_initial_schema.sql](#). Core tables:

Table	Key columns	Relations / notes
profiles	id UUID PK -> auth.users.id, full_name	1:1 with auth.users; auto-created by handle_new_user() trigger
videos	id, user_id, filename, storage_path, mime/file_size/duration/fps/w/h, status	status in UPLOADED PROCESSING COMPLETED FAILED DELETED; idx(user_id, created_at, status)
analyses	id, video_id, user_id, exercise, confidence, total/correct/incorrect_reps, form_accuracy, range_of_motion, movement_consistency, overall_score, status, model_metadata JSONB	13-stage status enum QUEUED..COMPLETED/FAILED; idx(user_id, video_id, status)
form_errors	id, analysis_id, error_type, severity LOW MED HIGH, rep_number, timestamp_seconds	Per-rep/per-timestamp faults; idx(analysis_id)
analysis_metrics	id, analysis_id, metric_name, metric_value, unit	Time-series/telemetry points; idx(analysis_id)
model_versions	model_name, model_type, version, training_dataset, feature_schema_version, metrics JSONB, artifact_path, status	UNIQUE(model_name, version)
reps (local/SQLite + API payload)	analysis_id, rep_number, durations, min/max/rom, form_errors, score	Per-rep telemetry surfaced by the API; mirror in Supabase as analysis_metrics rows

Triggers: handle_updated_at() keeps updated_at fresh on profiles/videos/analyses; on_auth_user_created seeds profiles.

3.3 Row Level Security [PROJECT]

RLS is enabled on all user tables. Owner-only pattern plus service-role bypass for the Celery worker (service_role skips RLS by design):

```
-- Profiles: users see only themselves
CREATE POLICY "Users can view own profile" ON public.profiles FOR SELECT
  USING (auth.uid() = id);
-- Videos / analyses: owner-only CRUD
CREATE POLICY "Users can view own videos" ON public.videos FOR SELECT
  USING (auth.uid() = user_id);
-- Child tables via parent ownership
CREATE POLICY "Users can view own form errors" ON public.form_errors FOR SELECT
  USING (EXISTS (SELECT 1 FROM public.analyses a
    WHERE a.id = form_errors.analysis_id AND a.user_id = auth.uid()));
```

3.4 Query cookbook (copy-paste) [PROJECT queries; REFERENCE patterns]

CRUD

```
-- Create analysis row
INSERT INTO public.analyses (video_id, user_id, exercise, status)
VALUES ('<vid>', auth.uid(), 'squat', 'QUEUED') RETURNING id;
-- Read own history (newest first, paginated)
SELECT * FROM public.analyses WHERE user_id = auth.uid()
ORDER BY created_at DESC LIMIT 20 OFFSET 0;
-- Update progress from worker (service_role client)
UPDATE public.analyses SET status='POSE_ESTIMATION', updated_at=NOW() WHERE id='<aid>';
-- Delete a video and its analyses (cascades via ON DELETE CASCADE)
DELETE FROM public.videos WHERE id='<vid>' AND user_id = auth.uid();
```

Joins: full analysis detail

```
SELECT a.id, a.exercise, a.overall_score, v.filename, v.duration_seconds,
       json_agg(json_build_object('error', f.error_type, 'rep', f.rep_number,
                                   't', f.timestamp_seconds)) AS faults
FROM public.analyses a JOIN public.videos v ON v.id = a.video_id
LEFT JOIN public.form_errors f ON f.analysis_id = a.id
WHERE a.user_id = auth.uid() AND a.id = '<aid>'
GROUP BY a.id, v.filename, v.duration_seconds;
```

Aggregation: dashboard stats

```
SELECT count(*) AS workouts, avg(overall_score) AS avg_score,
       sum(total_reps) AS reps, exercise, count(*) AS n
FROM public.analyses WHERE user_id = auth.uid() GROUP BY exercise;
```

Upsert + transactional batch (supabase-py)

```
# Upsert profile (REFERENCE)
db.table('profiles').upsert({'id': uid, 'full_name': name}, on_conflict='id').execute()
# Atomic multi-write via Postgres function (REFERENCE - wrap in one RPC call)
db.rpc('save_analysis_bundle', {'p_analysis': {...}, 'p_errors': [...]}).execute()
```

Full-text search + stored function [REFERENCE]

```
-- Search past feedback (requires a tsvector index; add in a new migration)
SELECT id, exercise FROM public.analyses
WHERE user_id = auth.uid() AND to_tsvector('english', model_metadata->'summary')
@@ plainto_tsquery('english', 'knee valgus') LIMIT 20;
-- Stored function: per-user weekly volume
CREATE OR REPLACE FUNCTION public.weekly_volume(p_uid uuid)
RETURNS TABLE (week timestampz, workouts bigint, reps bigint) LANGUAGE sql SECURITY DEFINER AS
$$
SELECT date_trunc('week', created_at), count(*), sum(total_reps) FROM public.analyses
WHERE user_id = p_uid GROUP BY 1 ORDER BY 1; $$
```

Realtime subscription [REFERENCE]

```
// Subscribe to your own analysis progress (supabase-js)
supa.channel('analysis-<aid>')
.on('postgres_changes', { event: 'UPDATE', schema: 'public', table: 'analyses',
  filter: 'id=eq.<aid>' }, (p) => renderStage(p.new.status)).subscribe()
```

The shipped app streams progress over SSE (GET /api/v1/analysis/{id}/stream) instead of Supabase Realtime; the snippet above is the migration path if you move status writes into Supabase.

3.5 Authentication [PROJECT state + REFERENCE path]

Current state [PROJECT]: demo-grade JWT (PyJWT, HS256, 24 h expiry) with automatic demo-user fallback and no password hashing - acceptable for a viva demo, NOT for production with real users. **Migration path**

[REFERENCE]: enable Supabase Auth (email + Google/GitHub OAuth), verify its JWTs in FastAPI using the project JWKS, and map **sub** -> **user_id** so every existing RLS policy keeps working unchanged.

```
# Supabase Auth (browser, REFERENCE)
await supa.auth.signInWithOAuth({ provider: 'google' })
const { data: { session } } = await supa.auth.getSession() # session.access_token
# FastAPI verification sketch (REFERENCE - replace get_current_user)
import jwt
claims = jwt.decode(token, key=jwks_client.get_key(kid).key, algorithms=["ES256"],
  audience="authenticated", issuer=f"{SUPABASE_URL}/auth/v1")
user_id = claims["sub"] # equals profiles.id / RLS auth.uid()
```

3.6 Storage [PROJECT]

```
-- Buckets are created by the migration (both private):
-- exercise-videos (500 MB limit, video/*) | analysis-reports (50 MB, application/pdf)
-- Folder convention: exercise-videos/{user_id}/{video_id}/{file}
-- Owner-scoped policies, e.g.:
CREATE POLICY "Users can view own exercise videos" ON storage.objects FOR SELECT
  USING (bucket_id='exercise-videos' AND auth.uid()::text = (storage.foldername(name))[1]);
```

```
# Signed URL for private playback/download (backend, service_role client)
url = db.storage.from_('exercise-videos').create_signed_url(f'{uid}/{vid}/{fn}',
  3600)['signedURL']
# Upload from backend
db.storage.from_('analysis-reports').upload(f'{uid}/{aid}/report.pdf', pdf_bytes,
  {'content-type': 'application/pdf', 'upsert': True})
```

3.7 Edge Functions [REFERENCE - not used by this project]

```
# Create + deploy (Supabase CLI)
supabase functions new thumbnail # Deno/TypeScript
supabase functions deploy thumbnail --project-ref <ref>
# Invoke (append ?download=1 for media)
curl -H "Authorization: Bearer $ANON_KEY" https://<ref>.functions.supabase.co/thumbnail
```

Good fits: post-analysis webhooks, thumbnailing, PDF watermarking. Keep ML inference OUT of Edge Functions (no GPU, 10 s CPU limits, cold starts) - it belongs in the Celery worker.

4. Platform-Specific Production Deployment

4.1 Vercel - frontend only [PROJECT-applicable]

Setting	Value for this project
Framework preset	Vite
Build command	npm run build (repo frontend/)
Output directory	dist
Root directory	frontend
Env vars	VITE_API_BASE=https:// (bake at build time!)

```
# SPA fallback so /history, /analysis/:id survive refresh (vercel.json in frontend/)
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Domains: project Settings -> Domains -> add apex + www (Vercel provisions TLS). Previews deploy per-PR automatically; roll back via Deployments -> Promote a previous deployment. **Do NOT deploy the FastAPI/ML backend to Vercel:** 10 MB serverless payload cap, 60 s max duration, and ~1 GB ML images are fundamentally incompatible with 24 s+ pose-estimation jobs. Edge runtime is irrelevant here (no middleware used).

4.2 Render - full stack [PROJECT-applicable]

Render hosts the API + worker + Redis natively. Use a **render.yaml** blueprint (infrastructure as code):

```
# render.yaml (repo root, REFERENCE starter - adjust repo/branch)
services:
  - type: web
    name: fitness-api
    runtime: docker
    dockerfilePath: ./backend/Dockerfile
    healthCheckPath: /health
    envVars:
      - key: SUPABASE_URL
        sync: false
      - key: SUPABASE_SERVICE_ROLE_KEY
        sync: false
      - key: REDIS_URL
        fromService: {name: fitness-redis, property: connectionString}
  - type: worker
    name: fitness-worker
    runtime: docker
    dockerfilePath: ./backend/Dockerfile
    dockerCommand: celery -A app.workers.celery_app.celery_app worker --loglevel=info
--concurrency=2
  - type: redis
    name: fitness-redis
    ipAllowList: [] # private to your services only
```

Persistent disk: only needed if you keep the SQLite fallback or on-disk uploads in production - mount at **/app/data** (Render disks cost extra and do NOT share between web and worker; prefer Supabase Storage + Postgres instead). Static frontend: serve dist/ as a Render Static Site with rewrite **/* -> /index.html**, or keep it on Vercel.

4.3 Alternatives [REFERENCE]

Platform	Fit for this project	Notes
Railway	Excellent full-stack fit	Dockerfile deploy, private networking, managed Redis + volumes; simplest compose parity
Fly.io	Good (global edge + volumes)	fly launch per service; volumes for data/models; careful with GPU-less CPU sizing for MediaPipe
AWS ECS/Fargate	Enterprise fit, most setup	ECR images + ALB + ElastiCache Redis + S3 for media; use when compliance demands a VPC

5. System Limitations & Constraints

5.1 Technical boundaries [PROJECT]

Boundary	Value / behavior
Max upload	500 MB / 300 s enforced in video_service + storage bucket cap
Analysis latency	~24 s per 18 s clip (CPU MediaPipe); scales linearly with frames
SSE stream timeout	nginx proxy_read_timeout 86400s; client must handle reconnects
Celery task limit	3600 s hard limit; concurrency 2 per worker
Cold start	First analysis pays TF/MediaPipe import (~10-20 s); keep worker warm

5.2 Platform restrictions [REFERENCE]

Platform	Restriction	Mitigation
Vercel serverless	10 MB payload; 10-60 s timeout	Frontend only; API elsewhere
Render free tier	Spins down after inactivity; slow cold starts	Paid instance or keep-alive ping for demos
Supabase free tier	500 MB DB, connection limits, 7-day log retention	Use Supavisor pooling (see 6.2); archive old analyses
Supabase connections	Direct Postgres ~60; pooler ~200+	Never open per-request raw psycopg2 pools from serverless

5.3 Rate limiting & quotas [PROJECT + REFERENCE]

Config guardrails exist (**AUTH 10/min**, **UPLOAD 5/hour**, **ANALYSIS 10/day**). Enforce at the API gateway (slowapi/Redis) and add per-user daily analysis caps in the analyses table to protect GPU/CPU budgets. Monitor Supabase MAU + storage egress weekly; archive raw videos to cold storage after N days, keeping only reports + metrics.

5.4 Security checklist [PROJECT + REFERENCE]

```
# Must-haves before any public deployment
# [ ] service-role key ONLY in backend env; anon key only in browser
# [ ] CORS_ORIGINS restricted to the real frontend domain (currently permissive for local dev)
# [ ] Validate MIME + magic bytes server-side; sanitize filenames (done in video_service)
# [ ] Rotate SUPABASE_SERVICE_ROLE_KEY + JWT secret quarterly; store in platform secret manager
# [ ] Upgrade auth to Supabase Auth (Section 3.5); add password hashing at minimum
# [ ] Backups: Supabase PITR (Pro) or nightly pg_dump (Section 6.4)
```

6. Database Operations & Query Optimization

6.1 Essential queries [PROJECT]

```
-- Filter + paginate history (keyset pagination - no OFFSET drift)
SELECT * FROM public.analyses WHERE user_id = auth.uid()
  AND (created_at, id) < ('<cursor_ts>', '<cursor_id>')
  ORDER BY created_at DESC, id DESC LIMIT 20;
-- Upsert model version registry row
INSERT INTO public.model_versions (model_name, model_type, version, metrics, status)
VALUES ('random_forest', 'sklearn', 'v3', '{"test_accuracy":0.9522}', 'ACTIVE')
ON CONFLICT (model_name, version) DO UPDATE SET metrics=EXCLUDED.metrics, status='ACTIVE';
-- Transactional batch: analysis + errors + metrics atomically
BEGIN;
INSERT INTO public.analyses (...) VALUES (...) RETURNING id;
INSERT INTO public.form_errors (analysis_id, ...) SELECT '<aid>', ... ;
INSERT INTO public.analysis_metrics (analysis_id, ...) SELECT '<aid>', ... ;
COMMIT;
```

6.2 Performance [PROJECT + REFERENCE]

```
-- Inspect a slow dashboard query
EXPLAIN (ANALYZE, BUFFERS) SELECT ... FROM public.analyses WHERE user_id = ...;
-- Recommended extras beyond the shipped indexes (add via migration)
CREATE INDEX IF NOT EXISTS idx_analyses_user_exercise ON public.analyses(user_id, exercise);
CREATE INDEX IF NOT EXISTS idx_form_errors_rep ON public.form_errors(analysis_id, rep_number);
```

Connection pooling: point serverless/many-replica workloads at the **Supavisor pooler (port 6543)**, keep the Celery worker on direct **5432** for long transactions. In supabase-py there is no client-side pool to tune; limit concurrency instead (worker --concurrency=2) and reuse one client per process.

6.3 Zero-downtime migrations [REFERENCE]

```
-- Pattern: additive change (safe to deploy while old code runs)
ALTER TABLE public.analysis ADD COLUMN IF NOT EXISTS grade TEXT;
-- Backfill in batches, then (later release) enforce NOT NULL / drop old column.
-- Destructive ops (rename, type change): expand -> migrate -> contract across 2 releases.
```

6.4 Backups & recovery [REFERENCE]

Method	RPO / RTO	When to use
Supabase PITR (Pro plan)	Minutes / minutes	Production: any-point restore
Nightly pg_dump via cron/CI	24 h / manual	Free tier: schema+data snapshot to versioned storage
Storage versioning	Per-object	Protect exercise-videos/analysis-reports from overwrites

```
# Nightly backup sketch (CI cron, REFERENCE)
pg_dump "$SUPABASE_DB_URL" -Fc -f backup-$(date +%F).dump
# restore: pg_restore -d "$SUPABASE_DB_URL" --clean backup-<date>.dump
```

7. Monitoring, Logging & Maintenance

7.1 Structured logging [PROJECT]

Implemented in **backend/app/core/logging.py**: JSON lines with **timestamp**, **level**, **logger**, **module**, **function**, **line** plus correlation extras **request_id**, **analysis_id**, **user_id**, **stage**, **duration**, **status**, **error** and exception capture. Add middleware that assigns a UUID **request_id** per HTTP request and passes **analysis_id** through every Celery stage (QUEUED -> POSE_ESTIMATION -> ... -> COMPLETED) so one ID traces upload to PDF.

```
# Propagate correlation explicitly (pattern used across services)
logger.info("Stage complete", extra={"request_id": rid, "analysis_id": aid, "stage":
"POSE_ESTIMATION"})
# Query one analysis end-to-end (Loki / CloudWatch Insights style)
{app="fitness-api"} | json | analysis_id="anl_abc123"
```

7.2 Error tracking [REFERENCE]

```
# Sentry (backend) - add to requirements + main.py startup
pip install sentry-sdk[fastapi]
import sentry_sdk
sentry_sdk.init(dsn=os.environ["SENTRY_DSN"], traces_sample_rate=0.1, environment="production")
# Frontend: @sentry/react with the same release tag; Celery: sentry-sdk[celery]
```

7.3 Uptime & alerting [PROJECT + REFERENCE]

Check	Target / threshold	Tool
GET /health == 200	< 2 s, 99.5%/mo	UptimeRobot / BetterUptime / Render health checks
Celery queue depth	Alert if > 10 or oldest > 15 min	Redis LLEN celery + worker heartbeat
Analysis p95 duration	Alert if > 3x baseline (~75 s)	Log-derived metric on processing_time_sec
5xx rate	Alert if > 1% over 5 min	Sentry + platform metrics
Supabase DB size / connections	Alert at 80% of plan	Supabase dashboard alerts

7.4 CI/CD pipeline [REFERENCE - starter, adapt paths]

```
# .github/workflows/ci.yml
name: ci
on: [push, pull_request]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python -m pytest tests/ -q
  frontend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: frontend } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm, cache-dependency-path: frontend/package-lock.json }
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npm run build
# Deploy jobs (Vercel/Render/Railway CLIs) run only on main after both pass.
```

Appendix A - Endpoint reference [PROJECT]

Method + path	Purpose
GET /health, GET /, GET /docs	Liveness, root links, Swagger UI
POST /api/v1/auth/register login; GET /me	Demo JWT auth
POST /videos/upload; GET /videos/; GET/DELETE /videos/{id}; GET .../stream	Video lifecycle
POST /videos/{id}/analyze; POST /analysis/start	Queue analysis (alias + SSE-oriented)
GET /analysis/{id}[/.../progress].../stream GET /analysis/	Result, polling, SSE, history
GET /reports/{id}/pdf	Generated PDF report
GET /models//metrics/{model_id}; POST /models/active	Model registry (real measured metrics)
GET /users/stats; GET PUT /users/profile	Dashboard aggregates, profile

Appendix B - Measured ML baselines [PROJECT]

Model	Task	Measured result (video-level splits)
Random Forest	Exercise recognition	Test acc 0.9522, macro F1 0.9516; weak class: bicep-curl recall ~0.42
XGBoost	Form faults (weak labels)	Val acc 0.810, macro F1 0.696; inference ~0.4 ms
Bi-LSTM (30-frame)	Temporal modeling	Val acc 0.839, val F1 0.850 (best-val checkpoint; train saturates)

Weak-label caveat: form labels are biomechanical-rule generated (backend/app/ml/features/form_labels.py), not human annotations. Do not present XGBoost metrics as human-ground-truth performance in academic reporting.